Class 11: Alphafold

Qiman A17912409

Custom analysis of resulting models

For tidiness we can move our AlphaFold results directory into our RStudio project directory.

```
results_dir <- "hivprdimer_23119.result"
```

```
#File names for all PDB models
pdb_files <- list.files(
  path = results_dir,
  pattern = "\\.pdb$",
  full.names = TRUE,
  recursive = TRUE
)

# Print our PDB file names
basename(pdb_files)
```

```
[1] "hivprdimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_2_seed_000.pdb"
[2] "hivprdimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_4_seed_000.pdb"
[3] "hivprdimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_1_seed_000.pdb"
[4] "hivprdimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_5_seed_000.pdb"
[5] "hivprdimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000.pdb"
```

```
library(bio3d)

# Read all data from Models
#  and superpose/fit coords
pdbs <- pdbaln(pdb_files, fit=TRUE, exefile="msa")
```

```
Reading PDB files:
hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_001_alphafold2_multi
hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_002_alphafold2_multi
hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_003_alphafold2_multi
hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_004_alphafold2_multi
hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_005_alphafold2_multi
.....

Extracting sequences

pdb/seq: 1   name: hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_0
pdb/seq: 2   name: hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_0
pdb/seq: 3   name: hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_0
pdb/seq: 4   name: hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_0
pdb/seq: 5   name: hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_0
```

```
pdbs
```

```
                              1        .         .         .         .         50
[Truncated_Name:1]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:2]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:3]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:4]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:5]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
                              **************************************************
                              1        .         .         .         .         50

                             51        .         .         .         .         100
[Truncated_Name:1]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:2]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:3]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:4]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:5]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
                              **************************************************
                             51        .         .         .         .         100

                            101        .         .         .         .         150
[Truncated_Name:1]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:2]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:3]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:4]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:5]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
```

```
                                 **************************************************
                               101        .         .         .         .         150

                               151        .         .         .         .       198
[Truncated_Name:1]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:2]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:3]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:4]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:5]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
                                 **********************************************
                               151        .         .         .         .       198

Call:
  pdbaln(files = pdb_files, fit = TRUE, exefile = "msa")

Class:
  pdbs, fasta

Alignment dimensions:
  5 sequence rows; 198 position columns (198 non-gap, 0 gap)

+ attr: xyz, resno, b, chain, id, ali, resid, sse, call
```

RMSD is a standard measure of structural distance between coordinate sets. We can use the `rmsd()` function to calculate the RMSD between all pairs models.

```
rd <- rmsd(pdbs, fit=T)
```

```
Warning in rmsd(pdbs, fit = T): No indices provided, using the 198 non NA positions
```

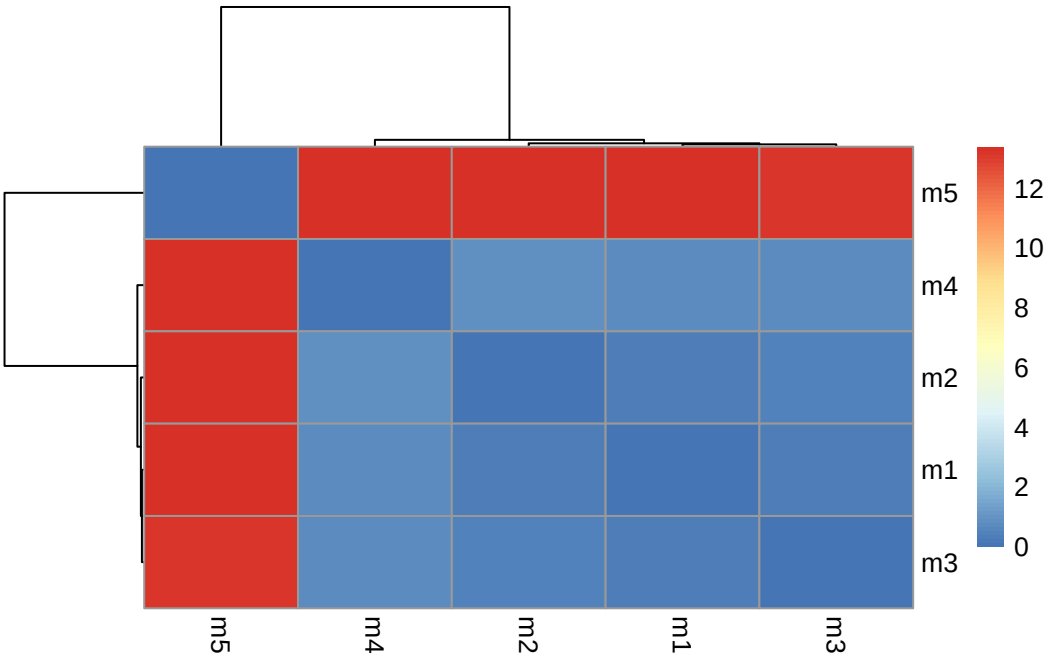
```
range(rd)
```

```
[1]  0.000 13.383
```

Draw a heatmap of these RMSD matrix values:

```
library(pheatmap)

colnames(rd) <- paste0("m",1:5)
rownames(rd) <- paste0("m",1:5)
pheatmap(rd)
```



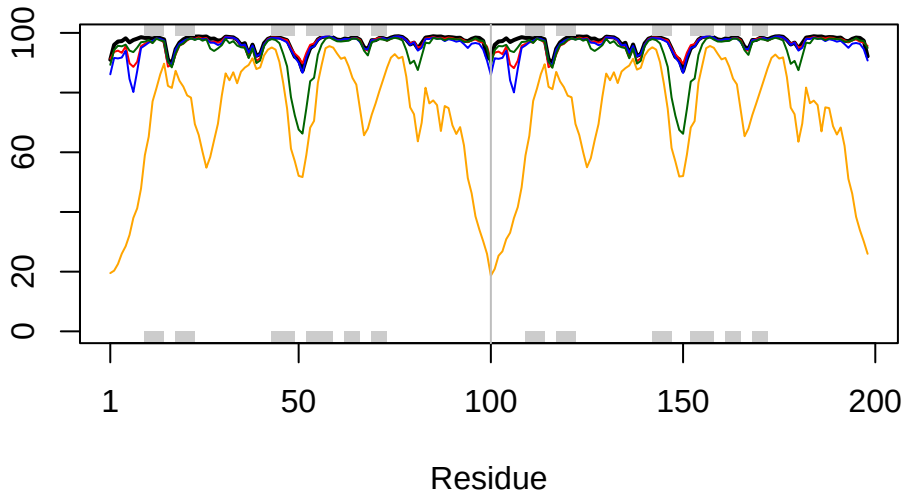
Now lets plot the pLDDT values across all models. Recall that this information is in the B-factor column of each model and that this is stored in our aligned `pdbs` object as `pdbs$b` with a row per structure/model.

```
# Read a reference PDB structure
pdb <- read.pdb("1hsg")
```

```
  Note: Accessing on-line PDB file
```

You could optionally obtain secondary structure from a call to `stride()` or `dssp()` on any of the model structures.

```
plotb3(pdbs$b[1,], typ="l", lwd=2, sse=pdb)
points(pdbs$b[2,], typ="l", col="red")
points(pdbs$b[3,], typ="l", col="blue")
points(pdbs$b[4,], typ="l", col="darkgreen")
points(pdbs$b[5,], typ="l", col="orange")
abline(v=100, col="gray")
```



We can improve the superposition/fitting of our models by finding the most consistent "rigid core" common across all the models. For this we will use the `core.find()` function:

```
core <- core.find(pdbs)
```

```
 core size 197 of 198  vol = 77.208
 core size 196 of 198  vol = 71.848
 core size 195 of 198  vol = 67.541
 core size 194 of 198  vol = 63.71
 core size 193 of 198  vol = 61.861
 core size 192 of 198  vol = 60.308
 core size 191 of 198  vol = 58.784
 core size 190 of 198  vol = 57.389
 core size 189 of 198  vol = 55.685
 core size 188 of 198  vol = 54.014
 core size 187 of 198  vol = 52.569
 core size 186 of 198  vol = 50.971
 core size 185 of 198  vol = 49.689
 core size 184 of 198  vol = 48.413
 core size 183 of 198  vol = 47.266
 core size 182 of 198  vol = 46.234
 core size 181 of 198  vol = 45.398
 core size 180 of 198  vol = 44.637
```

```
core size 179 of 198  vol = 43.693
core size 178 of 198  vol = 42.92
core size 177 of 198  vol = 42.248
core size 176 of 198  vol = 41.716
core size 175 of 198  vol = 41.449
core size 174 of 198  vol = 41.384
core size 173 of 198  vol = 41.231
core size 172 of 198  vol = 41.575
core size 171 of 198  vol = 41.637
core size 170 of 198  vol = 41.475
core size 169 of 198  vol = 41.168
core size 168 of 198  vol = 41.07
core size 167 of 198  vol = 41.088
core size 166 of 198  vol = 41.031
core size 165 of 198  vol = 40.853
core size 164 of 198  vol = 40.393
core size 163 of 198  vol = 39.786
core size 162 of 198  vol = 38.994
core size 161 of 198  vol = 38.39
core size 160 of 198  vol = 37.564
core size 159 of 198  vol = 37.047
core size 158 of 198  vol = 36.336
core size 157 of 198  vol = 35.74
core size 156 of 198  vol = 35.192
core size 155 of 198  vol = 34.348
core size 154 of 198  vol = 33.981
core size 153 of 198  vol = 33.487
core size 152 of 198  vol = 33.007
core size 151 of 198  vol = 31.8
core size 150 of 198  vol = 31.637
core size 149 of 198  vol = 31.041
core size 148 of 198  vol = 30.793
core size 147 of 198  vol = 30.105
core size 146 of 198  vol = 29.603
core size 145 of 198  vol = 29.215
core size 144 of 198  vol = 28.655
core size 143 of 198  vol = 28.061
core size 142 of 198  vol = 27.449
core size 141 of 198  vol = 26.852
core size 140 of 198  vol = 26.316
core size 139 of 198  vol = 25.023
core size 138 of 198  vol = 24.36
core size 137 of 198  vol = 23.612
```

```
core size 136 of 198  vol = 22.798
core size 135 of 198  vol = 22.014
core size 134 of 198  vol = 20.703
core size 133 of 198  vol = 19.835
core size 132 of 198  vol = 18.908
core size 131 of 198  vol = 18.34
core size 130 of 198  vol = 17.941
core size 129 of 198  vol = 17.187
core size 128 of 198  vol = 16.331
core size 127 of 198  vol = 15.498
core size 126 of 198  vol = 14.937
core size 125 of 198  vol = 14.248
core size 124 of 198  vol = 13.274
core size 123 of 198  vol = 12.489
core size 122 of 198  vol = 11.585
core size 121 of 198  vol = 10.702
core size 120 of 198  vol = 10.161
core size 119 of 198  vol = 9.526
core size 118 of 198  vol = 8.878
core size 117 of 198  vol = 8.212
core size 116 of 198  vol = 7.63
core size 115 of 198  vol = 7.35
core size 114 of 198  vol = 7.017
core size 113 of 198  vol = 6.691
core size 112 of 198  vol = 6.392
core size 111 of 198  vol = 5.952
core size 110 of 198  vol = 5.709
core size 109 of 198  vol = 5.5
core size 108 of 198  vol = 5.275
core size 107 of 198  vol = 4.928
core size 106 of 198  vol = 4.78
core size 105 of 198  vol = 4.561
core size 104 of 198  vol = 4.2
core size 103 of 198  vol = 3.89
core size 102 of 198  vol = 3.722
core size 101 of 198  vol = 3.642
core size 100 of 198  vol = 3.59
core size 99 of 198  vol = 3.287
core size 98 of 198  vol = 3.029
core size 97 of 198  vol = 2.776
core size 96 of 198  vol = 2.491
core size 95 of 198  vol = 2.076
core size 94 of 198  vol = 1.794
```

```
 core size 93 of 198  vol = 1.692
 core size 92 of 198  vol = 1.434
 core size 91 of 198  vol = 1.246
 core size 90 of 198  vol = 1.183
 core size 89 of 198  vol = 0.958
 core size 88 of 198  vol = 0.896
 core size 87 of 198  vol = 0.843
 core size 86 of 198  vol = 0.804
 core size 85 of 198  vol = 0.702
 core size 84 of 198  vol = 0.397
 FINISHED: Min vol ( 0.5 ) reached
```

```
core.inds <- print(core, vol=0.5)
```

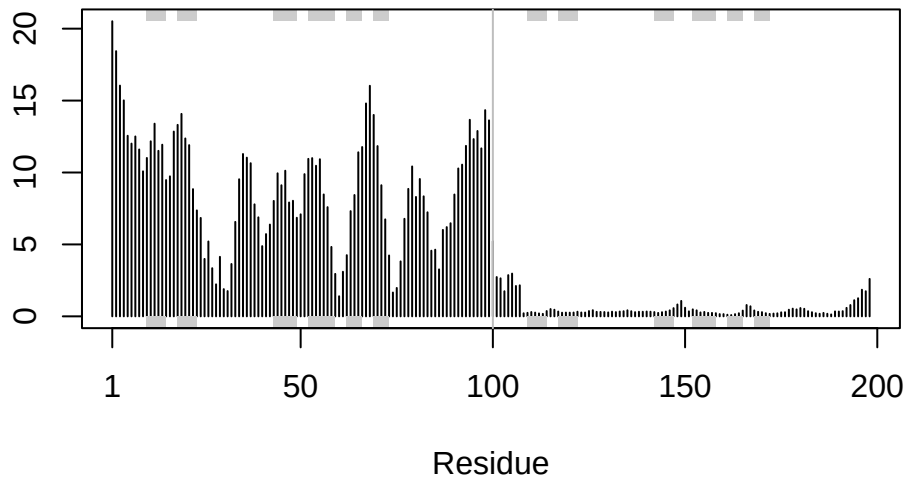
```
# 85 positions (cumulative volume <= 0.5 Angstrom^3)
  start end length
1     9  46     38
2    52  52      1
3    54  78     25
4    80  97     18
```

```
xyz <- pdbfit(pdbs, core.inds, outpath="corefit_structures")
```

Now we can examine the RMSF between positions of the structure. RMSF is an often used measure of conformational variance along the structure:

```
rf <- rmsf(xyz)

plotb3(rf, sse=pdb)
abline(v=100, col="gray", ylab="RMSF")
```



Independent of the 3D structure, AlphaFold produces an output called **Predicted Aligned Error (PAE)**. This is detailed in the JSON format result files, one for each model structure.

Below we read these files and see that AlphaFold produces a useful inter-domain prediction for model 1 (and 2) but not for model 5 (or indeed models 3, 4, and 5):

```
library(jsonlite)

results_dir <- "hivprdimer_23119.result"

# List the model score JSON files only
pae_files <- list.files(
  path = results_dir,
  pattern = "scores_rank_.*model.*\\.json$",
  full.names = TRUE,
  recursive = TRUE
)

basename(pae_files)
```

```
[1] "hivprdimer_23119_scores_rank_001_alphafold2_multimer_v3_model_2_seed_000.json"
[2] "hivprdimer_23119_scores_rank_002_alphafold2_multimer_v3_model_4_seed_000.json"
[3] "hivprdimer_23119_scores_rank_003_alphafold2_multimer_v3_model_1_seed_000.json"
```

```
[4] "hivprdimer_23119_scores_rank_004_alphafold2_multimer_v3_model_5_seed_000.json"
[5] "hivprdimer_23119_scores_rank_005_alphafold2_multimer_v3_model_3_seed_000.json"
```

```
length(pae_files)
```

```
[1] 5
```

```
pae1 <- read_json(pae_files[1], simplifyVector = TRUE)
pae5 <- read_json(pae_files[5], simplifyVector = TRUE)

attributes(pae1)
```

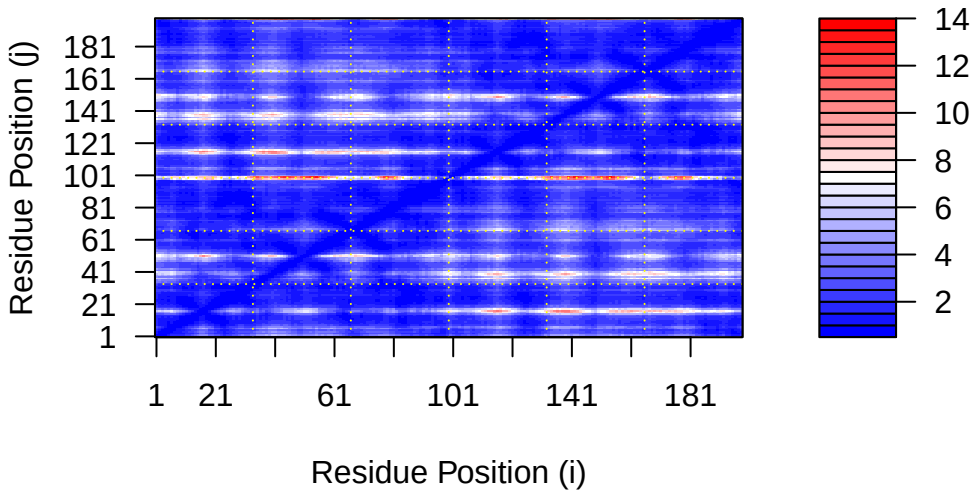
```
$names
[1] "plddt"   "max_pae" "pae"     "ptm"     "iptm"
```

```
# Per-residue pLDDT scores
#  same as B-factor of PDB..
head(pae1$plddt)
```

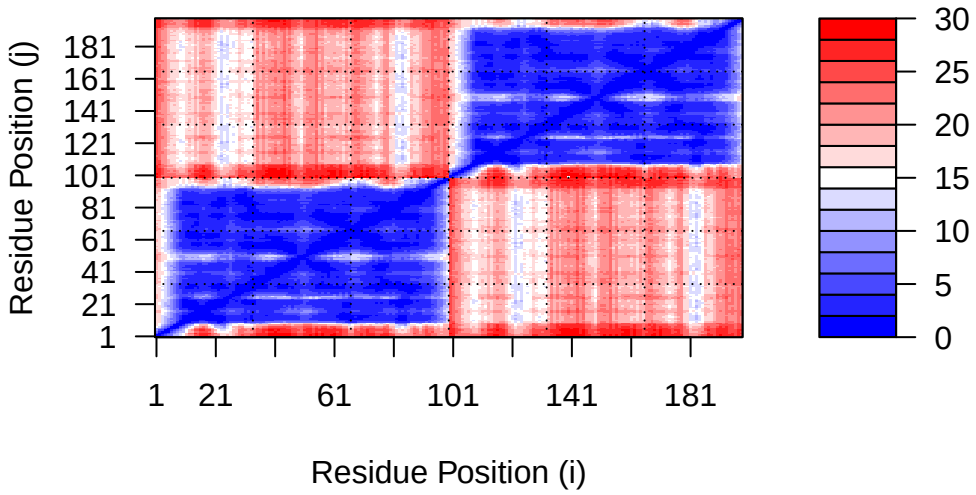
```
[1] 91.00 96.00 97.12 97.25 98.19 96.81
```

We can plot the N by N (where N is the number of residues) PAE scores with ggplot or with functions from the Bio3D package:

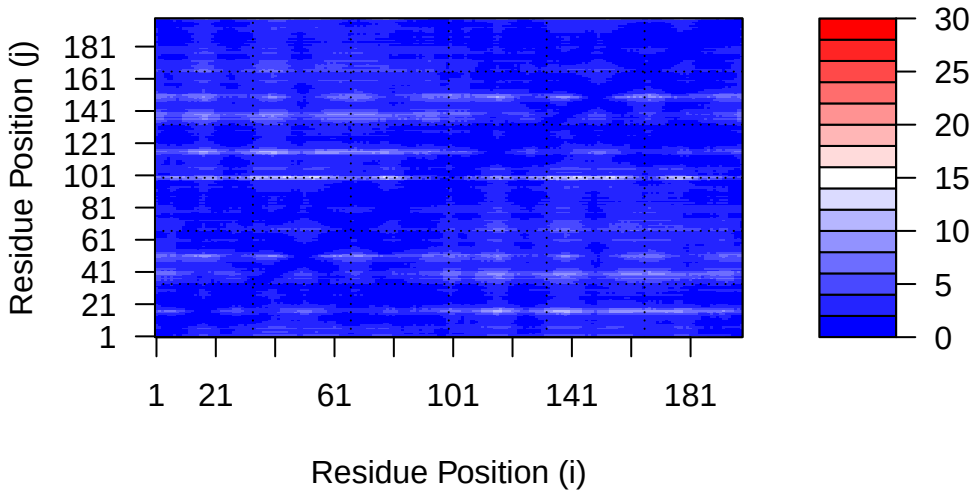
```
plot.dmat(pae1$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)")
```




```
plot.dmat(pae5$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```




```
plot.dmat(pae1$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```



Residue conservation from alignment file:

```
aln_file <- list.files(path=results_dir,
                       pattern=".a3m$",
                        full.names = TRUE,
                       recursive = TRUE)
aln_file
```

```
[1] "hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119.a3m"
[2] "hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_env/bfd.mgnify30.metaeuk30.sma
[3] "hivprdimer_23119.result/hivprdimer_23119/hivprdimer_23119_env/uniref.a3m"
```

```
aln <- read.fasta(aln_file[1], to.upper = TRUE)
```

```
[1] " ** Duplicated sequence id's: 101 **"
[2] " ** Duplicated sequence id's: 101 **"
```

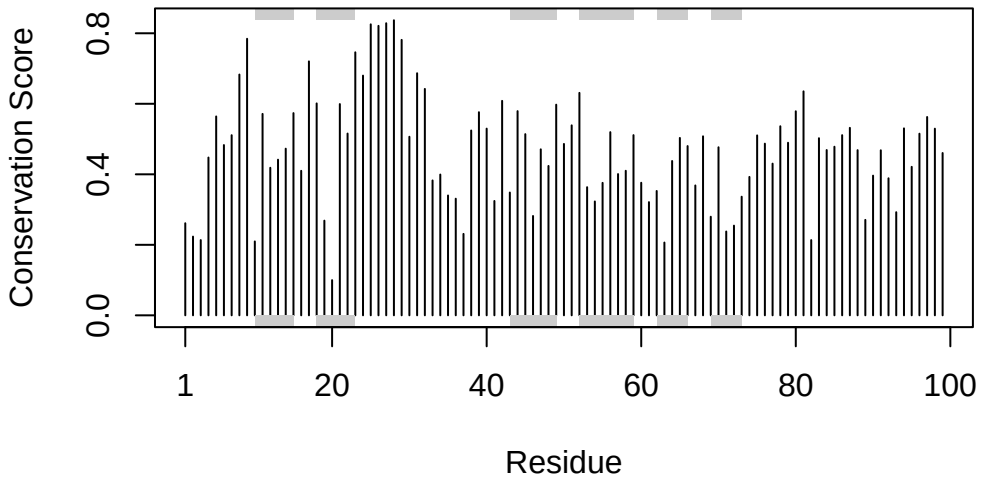
How many sequences are in this alignment?

```
dim(aln$ali)
```

```
[1] 5397  132
```

We can score residue conservation in the alignment with the `conserv()` function.

```
sim <- conserv(aln)
plotb3(sim[1:99], sse=trim.pdb(pdb, chain="A"),
       ylab="Conservation Score")
```



Note the conserved Active Site residues D25, T26, G27, A28. These positions will stand out if we generate a consensus sequence with a high cutoff value:

```
con <- consensus(aln, cutoff = 0.9)
con$seq
```

```
  [1] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
 [19] "-" "-" "-" "-" "-" "-" "D" "T" "G" "A" "-" "-" "-" "-" "-" "-" "-" "-"
 [37] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
 [55] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
 [73] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
 [91] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[109] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[127] "-" "-" "-" "-" "-" "-"
```

For a final visualization of these functionally important sites we can map this conservation score to the Occupancy column of a PDB file for viewing in molecular viewer programs such as Mol*, PyMol, VMD, chimera etc.

```
m1.pdb <- read.pdb(pdb_files[1])
occ <- vec2resno(c(sim[1:99], sim[1:99]), m1.pdb$atom$resno)
write.pdb(m1.pdb, o=occ, file="m1_conserv.pdb")
```